02 — Konsole

Erste Unterhaltung mit dem Computer: print und input

HTW Berlin – Wirtschaftsingenieurwesen

WiSe 2026/27

Agenda

1. Wozu eine Konsole? EVA wird konkret
2. Ausgaben mit `print(...)`
3. Unter der Haube: der Textstrom nach draußen
4. Eingaben mit `input(...)`
5. Unter der Haube: Anhalten, Warten, Text
6. Der Aha-Move: `input` direkt in `print`
7. **Klassiker: Begrüßungsdialog** – live programmiert

Lernziel: Den ersten echten Dialog zwischen Programm und Mensch bauen – und **verstehen**, was `print` und `input` dabei wirklich tun.

Heute: nach jedem Konzept *Live-Demo* (ich) und *Sofort ausprobieren* (Sie, Notebook 02).

Wozu eine Konsole?

Stummes Programm

```
"Eco-Sneaker"
"89.95"
```

→ Python arbeitet – aber wir **sehen nichts**.

Sprechendes Programm

```
print("Eco-Sneaker kostet 89.95 EUR")
```

→ jetzt erscheint etwas auf dem Bildschirm.

Die **Konsole** ist das Chat-Fenster zwischen Ihnen und dem Computer: Programme **schreiben** hinein, Sie **lesen** und **antworten**.

Rückblick E4: das EVA-Prinzip wird heute konkret

Aus der Computertechnik kennen Sie **E**ingabe → **V**erarbeitung → **A**usgabe. Heute bauen wir diesen Kreislauf zum ersten Mal **selbst**:

EVA-Schritt	Hardware (E4)	Heute in Python
Eingabe	Tastatur	`input(...)`
Verarbeitung	CPU / ALU	der Python-Interpreter
Ausgabe	Bildschirm	`print(...)`

Zwei Funktionen genügen für den kompletten EVA-Kreis – mehr brauchen wir heute nicht.

`print(...)` – der Mund des Programms

```
print("Hallo, Welt!")
print("Willkommen bei Veggie Soles!")
print('Vegane Sneaker, fair produziert.')
```

- `print` ist eine eingebaute **Funktion** – die Klammern `()` heißen “jetzt ausführen”
- Der Text dazwischen ist ein **String-Literal**: `"..."` oder `'...'`
- Die Anführungszeichen umrahmen den Text nur – sie erscheinen **nicht** in der Ausgabe
- Jedes `print(...)` endet automatisch mit einer **neuen Zeile**

Unter der Haube: was `print` wirklich tut

```
print("Eco-Sneaker", "89.95", "EUR")
```

Schritt	Was Python tut
1	nimmt **alle Argumente** zwischen den Klammern entgegen
2	verbindet sie mit je **einem Leerzeichen** zu einer Textzeile
3	hängt das **Zeilenende-Zeichen** `\n` an
4	schreibt alles in den **Ausgabestrom** (stdout)

`print` malt nicht auf den Bildschirm – es schreibt Text in einen **Strom**, den das Terminal anzeigt. Deshalb erscheinen Ausgaben exakt in der Reihenfolge der `print`-Aufrufe.

`print(...)` – Demo & Übung

▶ **Live-Demo** – `01_print_grundlagen.py`

✎ **Sofort ausprobieren** – Notebook 02, Kap. 2 (Mini-Übung): Namen ausgeben, drei Begrüßungen, Satz aus drei Teilen

`print(...)` – mehrere Werte und Leerzeilen

```
print("Eco-Sneaker", "89.95", "EUR")
print("Hemp-High", "109.00", "EUR")
print("")
print("Bambus-Boot", "135.50", "EUR")
```

- Mehrere Werte mit **Komma** – `print` setzt automatisch ein Leerzeichen dazwischen
- `print("")` oder `print()` erzeugt eine **leere Zeile**
- Damit lassen sich Ausgaben strukturieren: Überschrift, Trennlinie, Einträge, …

▶ **Live-Demo** – `02_print_mehrere.py`

Unter der Haube: \n – ein Steuerzeichen im Strom

```
print("Veggie Soles\nEco-Sneaker\nHemp-High")
```

Ausgabe – **drei** Zeilen aus **einem** `print`:

```
Veggie Soles
Eco-Sneaker
Hemp-High
```

- `\n` ist **ein einziges Zeichen** und bedeutet “neue Zeile” – es wird nie angezeigt, sondern vom Terminal **interpretiert**
- Genau dieses Zeichen hängt `print` selbst ans Ende – **deshalb** bricht jede Ausgabe um

▶ **Live-Demo** – `04_sonderzeichen.py`

`input(...)` – das Ohr des Programms

```
input("Wie heißen Sie? ")
```

1. Zeigt den **Prompt-Text** an (`"Wie heißen Sie? "`)
2. **Wartet**, bis Sie etwas tippen und **Enter** drücken
3. Liefert Ihre Eingabe als **Text** zurück

```
print(input("Wie heißen Sie? "))
```

→ fragt – und gibt die Antwort sofort wieder aus (Echo).

Unter der Haube: was `input` wirklich tut

```
print(input("Schuhgröße: "))
```

Schritt	Was passiert
1	`input` schreibt den Prompt `Schuhgröße:` in den Strom
2	das Programm **hält an** – der Cursor blinkt
3	Nutzer tippt 42 und drückt **Enter**
4	`input` liefert den **Text** `"42"` zurück – `print` gibt ihn aus

`input` liefert **immer Text** – auch `"42"` ist eine Zeichenfolge, keine Zahl. Rechnen können wir damit erst nach der Typumwandlung aus Notebook 05.

`input(...)` – Demo & Übung

▶ **Live-Demo** – `03_input_einfach.py`

✎ **Sofort ausprobieren** – Notebook 02, Kap. 3 (Mini-Übung): Lieblingsfarbe, zwei Fragen nacheinander, Mini-Dialog mit drei Fragen

Was uns heute (noch) fehlt

```
input("Wie heißen Sie? ")    # Antwort kommt an ...
print("... und ist weg.")    # ... unbenutzt verworfen
```

- Heute können wir Eingaben **nicht speichern** – das Werkzeug dafür heißt **Variable** und kommt in Notebook 04
- Jede Antwort muss **sofort** verwendet werden – genau dort, wo sie ankommt

Der Ausweg für heute: `input(...)` **direkt in** einen `print(...)` packen – nächste Folie.

Der Aha-Move: `input` direkt in `print`

```
print("Willkommen,", input("Ihr Name: "))
```

So läuft es ab:

```
Ihr Name: Anna
Willkommen, Anna
```

- `input(...)` steht **als Argument** mitten im `print(...)`
- Die Antwort landet direkt als **zweiter Wert** in der Ausgabe – ganz ohne Speichern
- Das Komma liefert wie immer das Leerzeichen zwischen den Werten

Unter der Haube: Auswertung von innen nach außen

```
print("Willkommen,", input("Ihr Name: "))
```

Schritt	Was Python tut
1	will `print` starten – aber das 2. Argument **fehlt noch**
2	also zuerst: `input` fragt, Programm wartet, Nutzer tippt `Anna`
3	`input` liefert `"Anna"` – jetzt sind **alle Argumente fertig**
4	erst jetzt läuft `print`: `Willkommen, Anna`

Regel: Python wertet Klammern **von innen nach außen** aus – eine Funktion startet erst, wenn alle ihre Argumente berechnet sind. **Deshalb** erscheint die Frage vor der Begrüßung.

Dialog – Demo & Übung

▶ **Live-Demo** – `05_dialog_kombiniert.py`

✎ **Sofort ausprobieren** – Notebook 02, Trainingsmaterial Aufg. 2 + 3: Echo-Programm, personalisierte Begrüßung

Klassiker: Begrüßungsdialog – die Aufgabe

Der Empfangstresen von Veggie Soles – komplett **ohne Variablen**:

- Ein kleines **Banner** aus mehreren `print`-Zeilen
- Das Programm **fragt nach dem Namen** (Prompt: `Ihr Name:`)
- Es **begrüßt persönlich** – die Antwort landet mitten im Satz
- Erlaubt sind nur `print`, `input` und String-Literale

So soll es laufen

```
================================
   VEGGIE SOLES  -  Willkommen
================================
Ihr Name: Anna
Willkommen, Anna !
```

Klassiker: Begrüßungsdialog – Live-Coding

▶ Live-Coding ab leerem Editor
Referenz: `90_klassiker_begruessung.py` – Teilziele:

1. Banner mit `print`-Zeilen aufbauen
2. `input` mit Prompt-Text stellen
3. `input` direkt in `print` einbetten
4. Mehrfach-Argumente für den Begrüßungssatz nutzen

✎ **Zum Selberlösen** – Handout *“Klassiker 02: Begrüßungsdialog”* (`Klassiker/02_Klassiker_Begruessung.pdf`). Vertiefung: Notebook 02, Trainingsmaterial Aufg. 4–6.

Cheat Card

Aufgabe	Syntax
Text ausgeben	`print("Text")`
Mehrere Werte	`print("a", "b")` – Leerzeichen automatisch
Leerzeile	`print()` oder `print("")`
Umbruch im Text	`"Zeile 1\nZeile 2"`
Eingabe lesen	`input("Frage? ")`
Eingabe sofort nutzen	`print("Hallo,", input("Name? "))`
Klassiker heute	Banner + `print("Willkommen,", input(...))`

Faustregel heute: `input` liefert **immer Text** – und ohne Variablen gilt: jede Antwort **dort verwenden, wo sie ankommt**.

Ausblick: Notebook 03 – Kommentare

Unser Klassiker funktioniert – aber **warum**, sieht man der Zeile nicht an:

```
print("Willkommen,", input("Ihr Name: "))
```

Wer das in drei Wochen liest, fragt: *Was passiert hier zuerst?*

- Nächste Woche: **Kommentare** – Notizen im Code (#), die Python ignoriert, aber Lesende retten
- Und in Notebook 04 lösen **Variablen** unser größtes Problem von heute: Antworten **speichern** und mehrfach verwenden

Heute geübt

- ✓ Die Konsole als Chat-Fenster – EVA-Prinzip konkret
- ✓ `print(...)`: Argumente → Textstrom (stdout) → Terminal
- ✓ Mehrere Werte per Komma, Leerzeilen, `\n` als Steuerzeichen
- ✓ `input(...)`: anhalten, warten, **immer Text** zurück
- ✓ Auswertung von innen nach außen: `input` direkt in `print`
- ✓ **Klassiker Begrüßungsdialog** live gesehen

✎ **Ihre Aufgaben bis zur nächsten Woche:**

- Klassiker **Begrüßungsdialog** eigenständig lösen (Handout)
- “Mini-Übungen” im Notebook 02 (Kap. 2 und 3)
- Trainingsmaterial: Begrüßung, Echo, Mini-Interview, Restaurant-Bestellung